

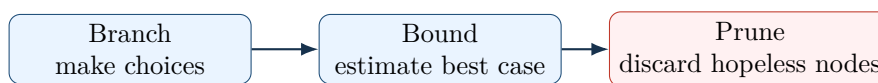
Branch and Bound Fundamentals

1 Core Idea

Branch and Bound

Branch and Bound is an exact optimization technique that searches a state-space tree while using bounds to avoid exploring branches that cannot improve the best solution already found.

It is used for optimization problems where the goal is to find the best feasible solution, not merely any feasible solution.

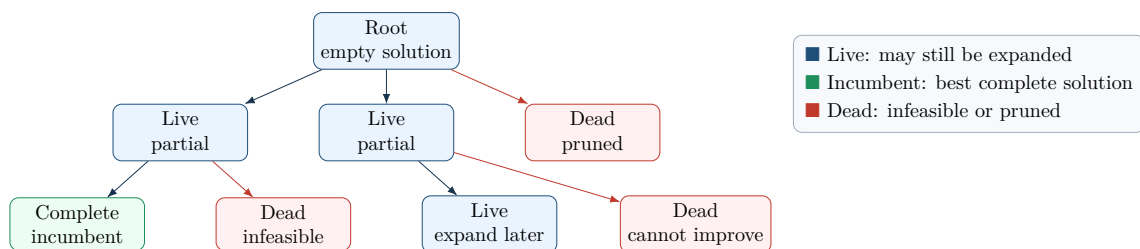


Not Greedy

Choosing the currently best-looking live node only controls the search order. Branch and Bound remains exact because other live nodes are kept until their bounds prove they can be discarded.

2 State-Space Tree

A state-space tree represents all possible decision paths. Branch and Bound keeps some generated nodes live and kills others using feasibility and bound tests.



Vocabulary

Live node	generated node that may still be expanded,
Dead node	completed, infeasible, expanded, or pruned node,
Incumbent	best complete feasible solution found so far.

3 Bounds

A bound is an optimistic estimate. It does not need to be the actual final value below the node; it must be good enough to prove when a branch cannot improve the incumbent.

3.1 Minimization Problems

For minimization, use a lower bound:

$$LB(N) = \text{least cost that node } N \text{ could possibly achieve.}$$

Let bestCost be the cost of the best complete feasible solution already found.

Minimization Pruning Rule

$$LB(N) \geq \text{bestCost} \implies \text{prune node } N.$$

Reason: even the most optimistic cost below N is not better than the current best cost.

3.2 Maximization Problems

For maximization, use an upper bound:

$$UB(N) = \text{greatest profit that node } N \text{ could possibly achieve.}$$

Let maxProfit be the profit of the best complete feasible solution already found.

Maximization Pruning Rule

$$UB(N) \leq \text{maxProfit} \implies \text{prune node } N.$$

Reason: even the most optimistic profit below N cannot beat the current best profit.

4 Lower Bound vs Upper Bound

Objective	Bound Used	Prune Condition
Minimize cost	Lower bound LB	$LB \geq \text{bestCost}$.
Maximize profit	Upper bound UB	$UB \leq \text{maxProfit}$.

Common Confusion

A lower bound is used for minimization, not maximization. An upper bound is used for maximization, not minimization. The bound must be optimistic in the direction of the objective.

5 Search Order

The live-node set may be managed in different ways.

Strategy	Live Node Removed Next
FIFO or BFS	The oldest generated live node.
LIFO or DFS	The most recently generated live node.
Least-cost search	The live node with the smallest lower bound.
Best-bound search	The live node with the most promising bound.

Breadth-first exploration is often introduced first, but many practical Branch and Bound algorithms choose the next live node by bound value.

6 Generic Algorithm

Branch and Bound Template

```
BranchAndBound(problem):
    create root node
    compute root bound
    liveNodes = {root}

    bestSolution = none
    bestValue = infinity      for minimization
                      = -infinity for maximization

    while liveNodes is not empty:
        node = remove one live node

        if node cannot improve bestValue:
            discard node
            continue

        if node is a complete feasible solution:
            update bestSolution and bestValue
            continue

        create child nodes

        for each child:
            if child is infeasible:
                discard child
            else:
                compute child bound
                if child can still improve bestValue:
                    add child to liveNodes

    return bestSolution
```

7 Small Numerical Illustration

Minimization Example

Suppose the current best complete solution has cost

$$\text{bestCost} = 18.$$

Three live nodes have lower bounds

$$LB(A) = 12, \quad LB(B) = 18, \quad LB(C) = 23.$$

Decision

For a minimization problem, prune when

$$LB \geq \text{bestCost}.$$

Node A :

$$LB(A) = 12 < 18,$$

so A may still lead to a better solution.

Node B :

$$LB(B) = 18 \geq 18,$$

so B cannot improve the incumbent.

Node C :

$$LB(C) = 23 \geq 18,$$

so C also cannot improve the incumbent.

Result

Keep A , prune B and C .

8 Correctness Intuition

Branch and Bound is correct when the bound is valid. A valid minimization lower bound never overstates how cheap a completion can be. A valid maximization upper bound never understates how profitable a completion can be.

1. Every feasible complete solution belongs to some branch of the state-space tree.
2. A branch is pruned only when its best possible optimistic value cannot improve the incumbent.
3. Therefore, no branch containing a better solution is discarded.

9 Complexity

Branch and Bound can greatly reduce practical search, but its worst-case complexity is still exponential or factorial for many optimization problems. The exact bound depends on the problem.

Problem Type	Typical Worst Case
Subset-style decisions	$\mathcal{O}(2^n)$.
Permutation-style decisions	$\mathcal{O}(n!)$.

Main Takeaway

Good bounds reduce the number of nodes expanded, but they do not automatically make an NP-hard problem polynomial in the worst case.

10 Final Review

Branch and Bound Essentials

Branch: split into decision subproblems.
Bound: estimate the best possible value below a node.
Incumbent: best complete feasible solution found so far.
Minimization: prune when $LB \geq \text{bestCost}$.
Maximization: prune when $UB \leq \text{maxProfit}$.

Assignment Problem by Branch and Bound

1 Problem Definition

Assignment Problem

There are n people and n jobs. Each person must receive exactly one job, each job must be assigned to exactly one person, and the total cost must be minimized.

For the example:

$$\text{People} = \{A, B, C, D\}, \quad \text{Jobs} = \{J_1, J_2, J_3, J_4\}.$$

The cost matrix is:

Person	J_1	J_2	J_3	J_4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

Objective

Find a one-to-one assignment minimizing

$$\sum_{i=1}^n c_{i,\pi(i)}$$

where $\pi(i)$ is the job assigned to person i .

2 Lower Bound

At a partial assignment, some people and jobs are already fixed. The lower bound is:

$$LB = \text{fixed cost} + \sum \text{minimum available cost for each unassigned person.}$$

This is a lower bound because it relaxes the future assignment problem: each remaining person is allowed to choose the cheapest still-available job independently. The true completion cannot cost less than this optimistic estimate.

3 Root Bound

At the root, no job is fixed. Take the minimum entry from each row:

$$\min(A) = 2, \quad \min(B) = 3, \quad \min(C) = 1, \quad \min(D) = 4.$$

Therefore,

$$LB_{\text{root}} = 2 + 3 + 1 + 4 = 10.$$

Root Node

$$LB_{\text{root}} = 10.$$

4 Branch on Person A

Create four branches by assigning A to each job. Once a job is chosen, it is unavailable to the remaining people.

Branch	Lower-bound calculation	LB
$A \rightarrow J_1$	$9 + \min(4, 3, 7) + \min(8, 1, 8) + \min(6, 9, 4)$	17
$A \rightarrow J_2$	$2 + \min(6, 3, 7) + \min(5, 1, 8) + \min(7, 9, 4)$	10
$A \rightarrow J_3$	$7 + \min(6, 4, 7) + \min(5, 8, 8) + \min(7, 6, 4)$	20
$A \rightarrow J_4$	$8 + \min(6, 4, 3) + \min(5, 8, 1) + \min(7, 6, 9)$	18

The smallest live lower bound is:

$$A \rightarrow J_2, \quad LB = 10.$$

So this node is expanded next.

5 Branch on Person B

Now $A \rightarrow J_2$ is fixed. Job J_2 is unavailable, so B may take J_1, J_3 , or J_4 .

Branch	Lower-bound calculation	LB
$B \rightarrow J_1$	$2 + 6 + \min(1, 8) + \min(9, 4)$	13
$B \rightarrow J_3$	$2 + 3 + \min(5, 8) + \min(7, 4)$	14
$B \rightarrow J_4$	$2 + 7 + \min(5, 1) + \min(7, 9)$	17

The best live node is:

$$A \rightarrow J_2, \quad B \rightarrow J_1, \quad LB = 13.$$

6 Complete the Best Branch

With $A \rightarrow J_2$ and $B \rightarrow J_1$ fixed, the remaining jobs are J_3 and J_4 . The remaining people are C and D .

Completion	Cost
$C \rightarrow J_3, D \rightarrow J_4$	$2 + 6 + 1 + 4 = 13$
$C \rightarrow J_4, D \rightarrow J_3$	$2 + 6 + 8 + 9 = 25$

The first completion gives a complete feasible solution:

$$\text{bestCost} = 13.$$

Incumbent

$$A \rightarrow J_2, \quad B \rightarrow J_1, \quad C \rightarrow J_3, \quad D \rightarrow J_4$$

with total cost

13.

7 Pruning Remaining Nodes

For minimization, prune a live node when

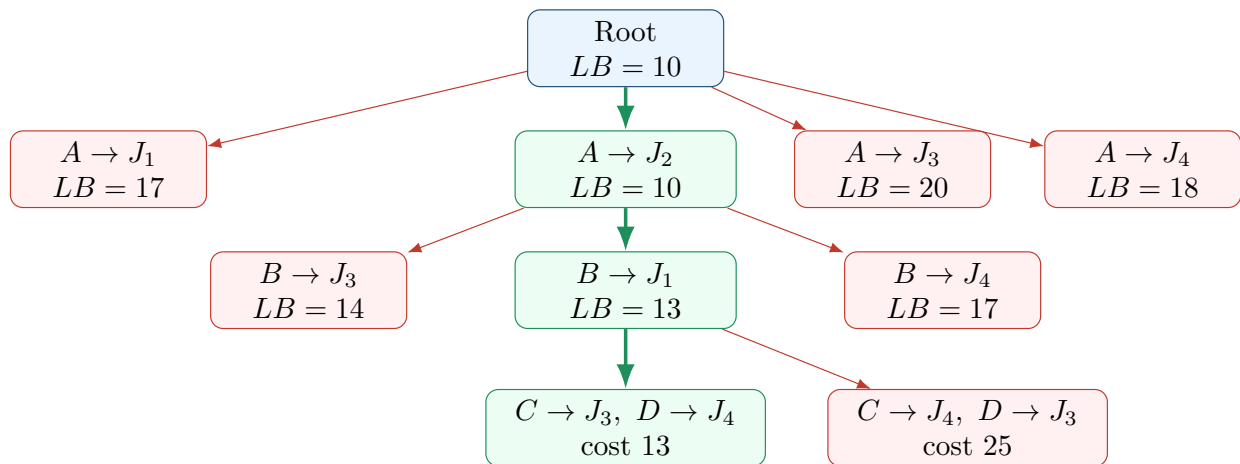
$$LB \geq \text{bestCost}.$$

Here $\text{bestCost} = 13$.

Remaining live node	LB	Decision
$A \rightarrow J_2, B \rightarrow J_3$	14	prune
$A \rightarrow J_2, B \rightarrow J_4$	17	prune
$A \rightarrow J_1$	17	prune
$A \rightarrow J_3$	20	prune
$A \rightarrow J_4$	18	prune

Every remaining live node has lower bound at least 13, so no branch can improve the incumbent.

8 State-Space Tree



9 Optimal Assignment

Person	Job	Cost
A	J_2	2
B	J_1	6
C	J_3	1
D	J_4	4

Minimum total cost = $2 + 6 + 1 + 4 = 13$.

10 Complexity

The worst-case number of possible assignments is the number of permutations of n jobs:

$$n!.$$

Therefore,

$$\boxed{\mathcal{O}(n!)}$$

is the worst-case time complexity for this Branch and Bound formulation.

Branch and Bound often expands far fewer nodes in practice, but the worst case remains factorial.

11 Final Review

Assignment Branch and Bound Essentials

$LB = \text{fixed cost} + \sum \text{row minima over available jobs.}$
For minimization, prune when $LB \geq \text{bestCost}$.
Best assignment: $A \rightarrow J_2, B \rightarrow J_1, C \rightarrow J_3, D \rightarrow J_4$.
Minimum cost = 13.

Hungarian Method

1 Purpose

Hungarian Method

The **Hungarian Method** solves the square assignment problem in polynomial time. It finds a minimum-cost one-to-one assignment between rows and columns.

For an $n \times n$ cost matrix $C = [c_{ij}]$, choose exactly one entry from each row and exactly one entry from each column so that the total cost is minimum.

Assignment Objective

$$\min_{\pi} \sum_{i=1}^n c_{i,\pi(i)}$$

where π is a permutation of the job columns.

2 Algorithm Steps

1. Subtract the minimum value of each row from every entry in that row.
2. Subtract the minimum value of each column from every entry in that column.
3. Cover all zeros using the minimum number of horizontal and vertical lines.
4. If the number of covering lines is n , choose independent zeros.
5. If fewer than n lines are needed, adjust the matrix and repeat.
6. Compute the final cost from the original cost matrix.

Independent Zeros

Independent zeros are zeros chosen so that no two selected zeros lie in the same row or the same column.

3 Original Matrix

Person	J_1	J_2	J_3	J_4	row min
A	6	5	7	7	5
B	12	8	9	3	3
C	7	6	9	10	6
D	6	3	7	8	3

The row minima are

5, 3, 6, 3.

4 Row Reduction

Subtract each row minimum from every entry of that row.

$$\begin{aligned}A &: (6, 5, 7, 7) - 5 = (1, 0, 2, 2), \\B &: (12, 8, 9, 3) - 3 = (9, 5, 6, 0), \\C &: (7, 6, 9, 10) - 6 = (1, 0, 3, 4), \\D &: (6, 3, 7, 8) - 3 = (3, 0, 4, 5).\end{aligned}$$

After row reduction:

Person	J_1	J_2	J_3	J_4
A	1	0	2	2
B	9	5	6	0
C	1	0	3	4
D	3	0	4	5

5 Column Reduction

Column minima after row reduction are

$$\min(J_1) = 1, \quad \min(J_2) = 0, \quad \min(J_3) = 2, \quad \min(J_4) = 0.$$

Subtract these values column by column.

Person	J_1	J_2	J_3	J_4
A	0	0	0	2
B	8	5	4	0
C	0	0	1	4
D	2	0	2	5

6 Zero Structure

After row and column reduction, zeros represent assignments that are cheapest relative to their row and column reductions. The goal is not to choose every zero. The goal is to choose n independent zeros.

Independent-Zero Requirement

For a 4×4 matrix, a complete assignment needs exactly four selected zeros:

one zero in each row and one zero in each column.

How to Choose Zeros Safely

Do not choose zeros randomly. A useful rule is to select forced zeros first.

1. Find a row with only one available zero and select that zero.
2. Cross out all other zeros in the selected zero's column.

- Find a column with only one available zero and select that zero.
- Cross out all other zeros in the selected zero's row.
- Repeat until each row and each column has exactly one selected zero.

If There Is a Tie

If several choices are possible, guessing may lead to a dead end. The safest interpretation is to treat zeros as a matching problem: match every row to a different zero-column. If a full matching is possible, the selected zeros give a valid optimal assignment.

For this matrix, the systematic selection is:

Step	Reason
Choose $B \rightarrow J_4$	Row B has only one zero.
Choose $D \rightarrow J_2$	Row D has only one zero.
Choose $C \rightarrow J_1$	After J_2 is used, row C 's safe available zero is J_1 .
Choose $A \rightarrow J_3$	Jobs J_1, J_2, J_4 are already used, so A takes J_3 .

Thus, one valid independent-zero selection is:

$$A \rightarrow J_3, \quad B \rightarrow J_4, \quad C \rightarrow J_1, \quad D \rightarrow J_2.$$

	J_1	J_2	J_3	J_4
A	0	0	0	2
B	8	5	4	0
C	0	0	1	4
D	2	0	2	5

Why This Selection Works

Check the selected rows:

$$A, \quad B, \quad C, \quad D$$

all appear exactly once.

Check the selected columns:

$$J_3, \quad J_4, \quad J_1, \quad J_2$$

also appear exactly once. Therefore, the selected zeros are independent.

Key Point

Other zeros may remain unselected. That is allowed. A valid assignment only needs one selected zero per row and one selected zero per column.

7 Cost from Original Matrix

The reduced matrix is used to find the assignment pattern. The final objective value must be computed from the original matrix.

Assignment	Original cost
$A \rightarrow J_3$	7
$B \rightarrow J_4$	3
$C \rightarrow J_1$	7
$D \rightarrow J_2$	3

$$\text{Minimum total cost} = 7 + 3 + 7 + 3 = 20.$$

8 Common Mistake: Feasible Does Not Mean Optimal

The following assignment is valid:

$$A \rightarrow J_1, \quad B \rightarrow J_4, \quad C \rightarrow J_3, \quad D \rightarrow J_2$$

It is feasible because every person gets exactly one job and no job is repeated.

However, feasibility alone does not prove optimality. Its original cost is

$$6 + 3 + 9 + 3 = 21.$$

Important Distinction

The Hungarian Method is not looking for just any feasible assignment. It is looking for the feasible assignment with minimum total cost.

The independent-zero assignment found above has cost

$$20.$$

Since

$$20 < 21,$$

the cost 21 assignment is valid, but it is not optimal.

9 Why Row and Column Reductions Work

Subtracting a constant from every entry of a row decreases the cost of every complete assignment by the same amount, because every complete assignment chooses exactly one entry from that row.

Similarly, subtracting a constant from every entry of a column decreases the cost of every complete assignment by the same amount, because every complete assignment chooses exactly one entry from that column.

Key Invariance

Row and column reductions change all complete assignment costs by the same total constant, so they do not change which assignment is optimal.

10 Complexity and Class

The Hungarian Method runs in polynomial time:

$$\mathcal{O}(n^3).$$

Therefore, the square assignment problem belongs to class P . The standard assignment problem is not NP-complete.

11 Final Review

Hungarian Method Essentials

Row reduction creates at least one zero in each row.
Column reduction creates at least one zero in each useful column.
Choose independent zeros systematically, not randomly.
Equivalently, find a full matching among zero positions.
Final cost is computed using the original matrix.
For this matrix, the optimum is 20.

0/1 Knapsack by Branch and Bound

1 Problem Definition

0/1 Knapsack

Given n items with weights w_i and values v_i , and a knapsack of capacity W , choose the most valuable subset of items whose total weight does not exceed W .

Each item is either selected completely or not selected at all:

$$x_i \in \{0, 1\},$$

where

$$x_i = \begin{cases} 1, & \text{item } i \text{ is included,} \\ 0, & \text{item } i \text{ is excluded.} \end{cases}$$

The goal is to maximize

$$\sum_{i=1}^n v_i x_i$$

subject to

$$\sum_{i=1}^n w_i x_i \leq W.$$

2 Example Data

The items are ordered in decreasing value-to-weight ratio:

$$\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \dots \geq \frac{v_n}{w_n}.$$

Item	Weight w_i	Value v_i	v_i/w_i
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

The knapsack capacity is

$$W = 10.$$

3 State-Space Tree

The state-space tree is a binary tree. At level i , the algorithm decides whether to include or exclude item i .

left branch = include the next item, right branch = exclude the next item.

Each node stores:

$$(w, v, ub),$$

where

w = current total weight, v = current total value, ub = upper bound on possible value.

4 Upper Bound Formula

For a node after deciding the first i items, suppose the next undecided item is $i + 1$. A simple upper bound is:

Upper Bound

$$ub = v + (W - w) \left(\frac{v_{i+1}}{w_{i+1}} \right).$$

This bound says: keep the value v already selected, then fill the remaining capacity $W - w$ using the best value per unit weight among the remaining ordered items.

Why This Is an Upper Bound

The formula is optimistic because it acts as if the remaining capacity can be filled fractionally using the next best ratio. The real 0/1 solution cannot be better than this optimistic estimate.

5 Root Node

At the root, no item has been selected:

$$w = 0, \quad v = 0.$$

The next item is item 1, whose ratio is 10. Hence,

$$ub = 0 + (10 - 0)(10) = 100.$$

Root

$$w = 0, \quad v = 0, \quad ub = 100.$$

6 Branch on Item 1

From the root, create two branches.

Node	Weight	Value	Upper-bound calculation
With item 1	4	40	$40 + (10 - 4)(6) = 76$
Without item 1	0	0	$0 + (10 - 0)(6) = 60$

The node with item 1 has the larger upper bound:

$$\boxed{76 > 60.}$$

So the best-first branch-and-bound algorithm expands that node next.

7 Branch on Item 2

Now item 1 is included:

$$w = 4, \quad v = 40.$$

Node	Weight	Value	Status
With item 2	$4 + 7 = 11$	$40 + 42 = 82$	infeasible because $11 > 10$
Without item 2	4	40	$ub = 40 + (10 - 4)(5) = 70$

The branch that includes item 2 is discarded immediately because it exceeds the capacity.

8 Branch on Item 3

The best live node is now:

$$w = 4, \quad v = 40, \quad ub = 70.$$

This means item 1 is included and item 2 is excluded.

Node	Weight	Value	Upper bound or status
With item 3	$4 + 5 = 9$	$40 + 25 = 65$	$ub = 65 + (10 - 9)(4) = 69$
Without item 3	4	40	$ub = 40 + (10 - 4)(4) = 64$

The branch with item 3 is more promising because

$$69 > 64.$$

9 Branch on Item 4

Now items 1 and 3 are included, while item 2 is excluded:

$$w = 9, \quad v = 65.$$

Node	Weight	Value	Status
With item 4	$9 + 3 = 12$	$65 + 12 = 77$	infeasible because $12 > 10$
Without item 4	9	65	feasible complete subset

The branch without item 4 gives a feasible solution:

$$\boxed{\{1, 3\}}$$

with total weight

$$4 + 5 = 9$$

and total value

$$40 + 25 = 65.$$

So the current best value becomes

$$\text{bestValue} = 65.$$

10 Pruning Remaining Nodes

For a maximization problem, a node is pruned when its upper bound is not better than the best feasible value already found.

Pruning Rule

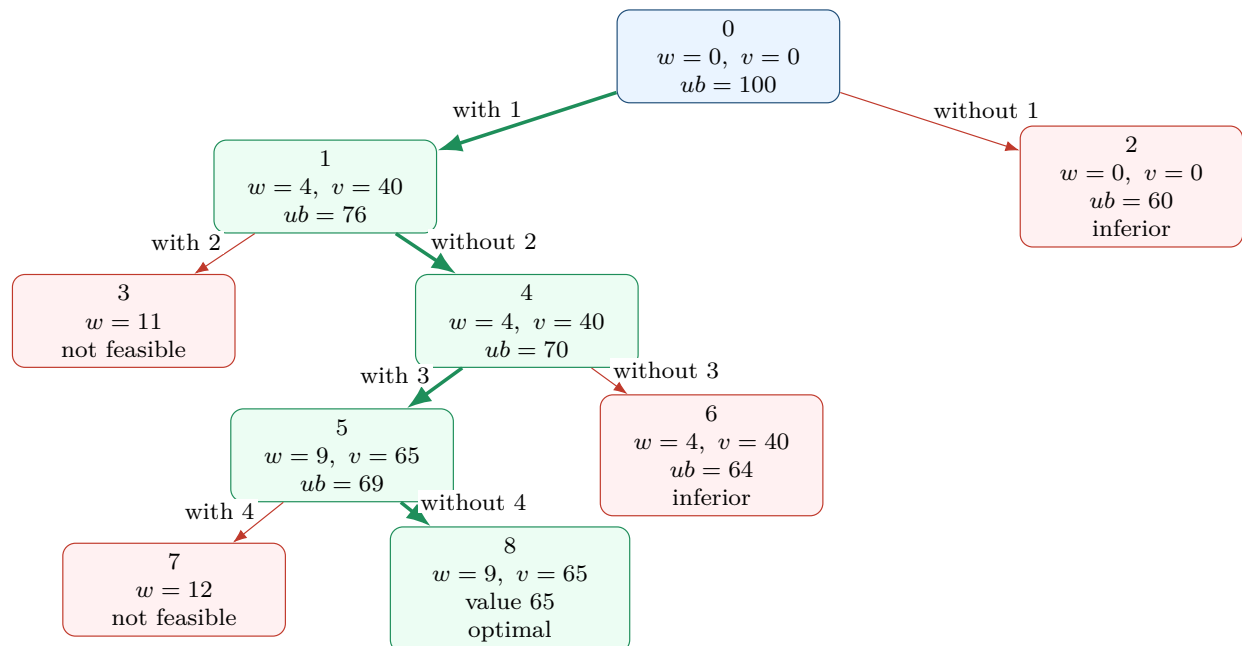
$$ub \leq \text{bestValue} \implies \text{prune the node.}$$

After finding $\text{bestValue} = 65$, the remaining live nodes are:

Node	Upper bound	Decision
Without item 1	60	prune
Without item 3 after choosing item 1	64	prune

Both nodes cannot improve the value 65, so they are terminated.

11 State-Space Tree Diagram



12 Optimal Solution

The optimal subset is:

$$\{1, 3\}.$$

The decision vector is:

$$(x_1, x_2, x_3, x_4) = (1, 0, 1, 0).$$

Total weight:

$$4 + 5 = 9 \leq 10.$$

Total value:

$$40 + 25 = 65.$$

Answer

$$\text{Maximum value} = 65.$$

13 Second Worked Problem

Now solve another 0/1 knapsack instance:

Original item	1	2	3	4
Value	10	10	12	18
Weight	2	4	6	9

with capacity

$$W = 15.$$

Sort by Value-to-Weight Ratio

Compute

$$\frac{v_i}{w_i}.$$

Original item	w_i	v_i	v_i/w_i
1	2	10	5
2	4	10	2.5
3	6	12	2
4	9	18	2

Items 3 and 4 have the same ratio. We keep the original order between them. Therefore, the ordered items are:

$$1, 2, 3, 4.$$

Root Bound

At the root,

$$w = 0, \quad v = 0.$$

The best ratio is item 1's ratio 5. So

$$ub = 0 + (15 - 0)(5) = 75.$$

Branch on Item 1

Node	Weight	Value	Upper-bound calculation
With item 1	2	10	$10 + (15 - 2)(2.5) = 42.5$
Without item 1	0	0	$0 + (15 - 0)(2.5) = 37.5$

The branch with item 1 is expanded first because

$$42.5 > 37.5.$$

Branch on Item 2

Now item 1 is included:

$$w = 2, \quad v = 10.$$

Node	Weight	Value	Upper-bound calculation
With item 2	$2 + 4 = 6$	$10 + 10 = 20$	$20 + (15 - 6)(2) = 38$
Without item 2	2	10	$10 + (15 - 2)(2) = 36$

The branch with item 2 is expanded next because

$$38 > 36.$$

Branch on Item 3

Now items 1 and 2 are included:

$$w = 6, \quad v = 20.$$

Node	Weight	Value	Upper-bound calculation
With item 3	$6 + 6 = 12$	$20 + 12 = 32$	$32 + (15 - 12)(2) = 38$
Without item 3	6	20	$20 + (15 - 6)(2) = 38$

Both branches have the same upper bound. Use a consistent tie rule; here we expand the branch with item 3 first.

Branch on Item 4 After Choosing Items 1, 2, and 3

Now

$$w = 12, \quad v = 32.$$

Node	Weight	Value	Status
With item 4	$12 + 9 = 21$	$32 + 18 = 50$	infeasible because $21 > 15$
Without item 4	12	32	feasible subset $\{1, 2, 3\}$

So the current best feasible value is

$\text{bestValue} = 32$

from subset

$\{1, 2, 3\}.$

Check the Other Tied Branch

The tied branch after item 3 was skipped is:

$$\{1, 2\}, \quad w = 6, \quad v = 20.$$

If item 4 is included:

$$w = 6 + 9 = 15, \quad v = 20 + 18 = 38.$$

This is feasible and improves the best value:

$$38 > 32.$$

Therefore,

$$\boxed{\text{bestValue} = 38}$$

with subset

$$\boxed{\{1, 2, 4\}}.$$

If item 4 is excluded, the value remains 20, which is worse.

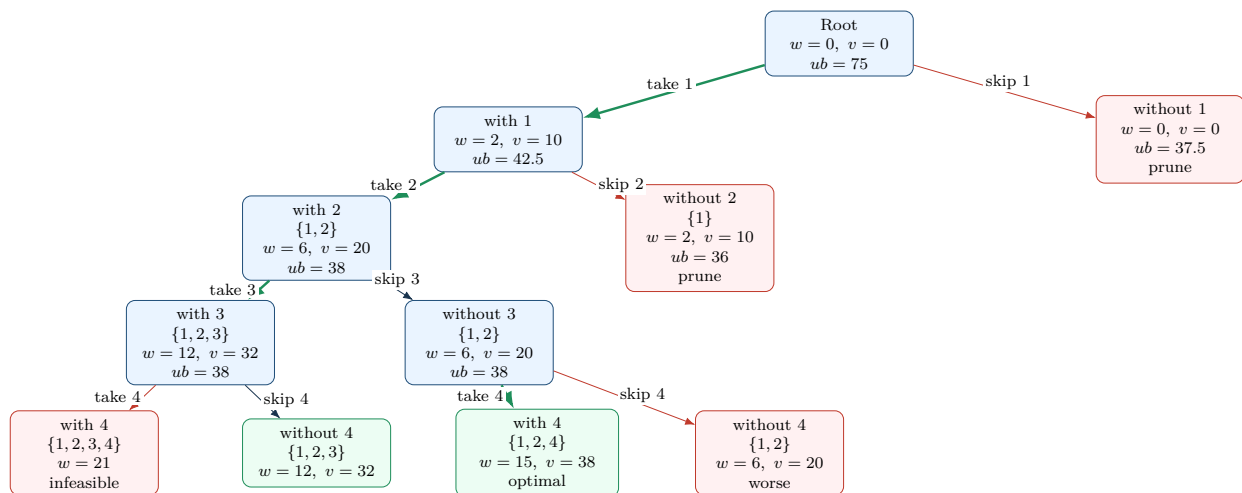
Prune Remaining Branches

The remaining live branches have upper bounds:

Branch	ub	Decision
Without item 1	37.5	prune because $37.5 < 38$
With item 1, without item 2	36	prune because $36 < 38$

No remaining branch can improve the current best value 38.

State-Space Tree for the Second Problem



Second Problem Answer

The optimal subset is

$$\{1, 2, 4\}.$$

Total weight:

$$2 + 4 + 9 = 15 \leq 15.$$

Total value:

$$10 + 10 + 18 = 38.$$

Second Problem Answer

Maximum value = 38.

14 Third Worked Problem

Now solve:

Original item	1	2	3	4
Weight	5	7	4	2
Value	30	28	20	24

with capacity

$$W = 12.$$

Sort by Value-to-Weight Ratio

Original item	w_i	v_i	v_i/w_i
1	5	30	6
2	7	28	4
3	4	20	5
4	2	24	12

So the branch-and-bound order is:

$$4, 1, 3, 2.$$

Sorted label	Original item	w	v	v/w
S_1	4	2	24	12
S_2	1	5	30	6
S_3	3	4	20	5
S_4	2	7	28	4

Item Numbering

From now on, branching uses the sorted labels S_1, S_2, S_3, S_4 . The final answer is converted back to the original item numbers.

Root and Main Branches

At the root,

$$w = 0, \quad v = 0, \quad ub = 0 + (12)(12) = 144.$$

Node	Weight	Value	Upper-bound calculation
With S_1	2	24	$24 + (12 - 2)(6) = 84$
Without S_1	0	0	$0 + (12)(6) = 72$

Expand the branch with S_1 , because

$$84 > 72.$$

Continue with S_2

Now S_1 is included:

$$w = 2, \quad v = 24.$$

Node	Weight	Value	Upper-bound calculation
With S_2	$2 + 5 = 7$	$24 + 30 = 54$	$54 + (12 - 7)(5) = 79$
Without S_2	2	24	$24 + (12 - 2)(5) = 74$

Expand the branch with S_2 , because

$$79 > 74.$$

Continue with S_3

Now S_1 and S_2 are included:

$$w = 7, \quad v = 54.$$

Node	Weight	Value	Upper-bound calculation
With S_3	$7 + 4 = 11$	$54 + 20 = 74$	$74 + (12 - 11)(4) = 78$
Without S_3	7	54	$54 + (12 - 7)(4) = 74$

Expand the branch with S_3 , because

$$78 > 74.$$

Decision on S_4

Now S_1, S_2, S_3 are included:

$$w = 11, \quad v = 74.$$

Node	Weight	Value	Status
With S_4	$11 + 7 = 18$	$74 + 28 = 102$	infeasible because $18 > 12$
Without S_4	11	74	feasible sorted subset $\{S_1, S_2, S_3\}$

So the current best value is

$$\text{bestValue} = 74$$

from original items

$$\{1, 3, 4\}.$$

Pruning

The remaining live branches cannot improve the best value:

Branch	<i>ub</i>	Decision
Without S_1	72	prune because $72 < 74$
With S_1 , without S_2	74	prune because $74 \leq 74$
With S_1, S_2 , without S_3	74	prune because $74 \leq 74$

Third Problem Answer

The optimal subset is

$$\{1, 3, 4\}.$$

Total weight:

$$5 + 4 + 2 = 11 \leq 12.$$

Total value:

$$30 + 20 + 24 = 74.$$

Third Problem Answer

$$\text{Maximum value} = 74.$$

15 Pseudocode

Best-First Knapsack Branch and Bound

```
KnapsackBranchAndBound(items, W):
    sort items by decreasing value/weight ratio
    root.weight = 0
    root.value = 0
    root.bound = upperBound(root)

    liveNodes = {root}
    bestValue = 0
    bestSubset = empty

    while liveNodes is not empty:
        node = remove live node with largest upper bound

        if node.bound <= bestValue:
            continue

        create child that includes the next item
```

```

    if child.weight <= W:
        if child.value > bestValue:
            bestValue = child.value
            bestSubset = child.subset
        child.bound = upperBound(child)
        if child.bound > bestValue:
            add child to liveNodes

    create child that excludes the next item
    child.bound = upperBound(child)
    if child.bound > bestValue:
        add child to liveNodes

return bestSubset and bestValue

```

16 Complexity

Each item creates two choices:

include or exclude.

Therefore, the worst-case search tree can still have

$$2^n$$

nodes.

Worst-case time = $\mathcal{O}(2^n)$.
--

Branch and Bound is useful because upper bounds often prune many nodes before the full tree is generated.

17 Final Review

0/1 Knapsack Branch and Bound Essentials

Order items by decreasing v_i/w_i. Each node stores w, v, and ub. $ub = v + (W - w)(v_{i+1}/w_{i+1})$. For maximization, prune when $ub \leq \text{bestValue}$. Best subset here is $\{1, 3\}$ with value 65.

Traveling Salesman Problem by Branch and Bound

1 Problem Definition

Traveling Salesman Problem

Given n cities and the distance between every pair of cities, find a shortest tour that starts from one city, visits every other city exactly once, and returns to the starting city.

A tour is also called a Hamiltonian circuit. For cities

$$\{a, b, c, d, e\},$$

one possible tour is

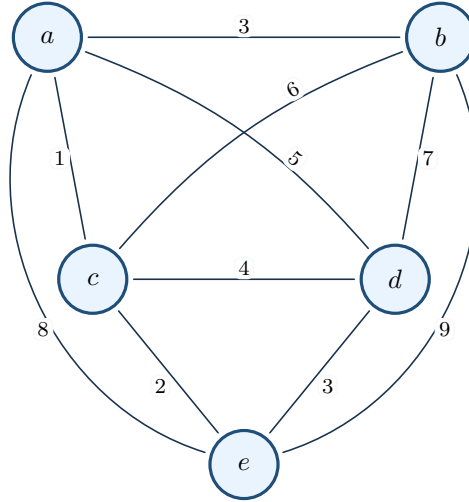
$$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a.$$

The objective is to minimize the total length of the tour:

$$\text{tour length} = \sum \text{lengths of the selected tour edges.}$$

2 Example Graph

The example uses the following undirected weighted graph.



3 Why Branch and Bound Is Useful

If the starting city is fixed as a , then a brute-force search checks

$$(n - 1)!$$

possible orders of the remaining cities. For 5 cities, this is

$$4! = 24$$

tours.

Branch and Bound avoids checking every tour by removing branches that cannot beat the best tour already found.

Main Idea

Build tours step by step. For each partial tour, compute a lower bound. If the lower bound is already not better than the best complete tour found so far, prune that branch.

4 Lower Bound for TSP

In any Hamiltonian circuit, every city has degree 2, because one edge enters the city and one edge leaves it.

Therefore, a simple lower bound is obtained by taking the two smallest incident edge lengths for every city.

TSP Lower Bound

For each city i , let s_i be the sum of the two smallest edge lengths incident with city i . Then

$$LB = \left\lceil \frac{\sum_{i=1}^n s_i}{2} \right\rceil.$$

We divide by 2 because every edge is counted at both of its endpoints.

5 Root Lower Bound

Find the two smallest incident edge lengths for each city.

City	Two smallest incident edges	Sum
a	$ac = 1, ab = 3$	$1 + 3 = 4$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

So,

$$\sum s_i = 4 + 9 + 3 + 7 + 5 = 28.$$

Therefore,

$$LB_{\text{root}} = \left\lceil \frac{28}{2} \right\rceil = 14.$$

6 Bound When Some Edges Are Forced

Sometimes a branch already forces some edges to be included in the tour. Then the bound must respect those required edges.

For example, suppose the partial tour begins with

$$a \rightarrow d.$$

Then the edge ad must be included at both city a and city d .

$$\begin{aligned} LB &= \left\lceil \frac{(1+5) + (3+6) + (1+2) + (3+5) + (2+3)}{2} \right\rceil \\ &= \left\lceil \frac{6+9+3+8+5}{2} \right\rceil \\ &= \left\lceil \frac{31}{2} \right\rceil \\ &= 16. \end{aligned}$$

Meaning of the Bound

A lower bound is not always the actual tour length. It is an optimistic estimate. If even this optimistic estimate is too large, the branch cannot contain the optimal tour.

7 Branching Strategy

To avoid generating the same undirected tour many times, fix the starting city as a . Then branch by choosing the next city after a .

The first-level branches are:

$$a, b \quad a, c \quad a, d \quad a, e.$$

For each branch, compute a lower bound and expand the most promising live node.

Node	Meaning	Bound or status
a, b	tour begins $a \rightarrow b$	14
a, c	tour begins $a \rightarrow c$	14, skipped only by symmetry rule
a, d	tour begins $a \rightarrow d$	16
a, e	tour begins $a \rightarrow e$	19

How the Table Bounds Are Calculated

For each branch, the lower bound uses the same rule:

$$LB = \left\lceil \frac{\sum s_i}{2} \right\rceil,$$

where s_i is the sum of two required or cheapest incident edges for city i . If a branch forces an edge, that forced edge must be included in the two edges for its endpoint cities.

Branch a, b :

The branch starts with edge $ab = 3$. This edge is already one of the two cheapest choices for both a and b , so the root bound stays the same.

City	two edges used in bound	s_i
a	$ac = 1, ab = 3$	$1 + 3 = 4$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

$$LB = \left\lceil \frac{4 + 9 + 3 + 7 + 5}{2} \right\rceil = \left\lceil \frac{28}{2} \right\rceil = 14.$$

Branch a, c , if it is not skipped:

The branch starts with edge $ac = 1$. This branch is valid. If we decide not to use symmetry skipping, then its lower bound is calculated exactly like the other branches. Since ac is already one of the two cheapest incident edges for both a and c , the lower bound again remains the same as the root bound.

City	two edges used in bound	s_i
a	$ac = 1, ab = 3$	$1 + 3 = 4$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

$$LB = \left\lceil \frac{4 + 9 + 3 + 7 + 5}{2} \right\rceil = \left\lceil \frac{28}{2} \right\rceil = 14.$$

Safe Interpretation

The branch a, c is not removed by the lower-bound test. If no symmetry-breaking rule is stated, a, c should be treated as a normal live branch with lower bound 14.

Branch a, d :

The branch forces edge $ad = 5$. Therefore, ad must be counted for city a and city d .

City	two edges used in bound	s_i
a	$ac = 1, ad = 5$	$1 + 5 = 6$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, da = 5$	$3 + 5 = 8$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

$$LB = \left\lceil \frac{6 + 9 + 3 + 8 + 5}{2} \right\rceil = \left\lceil \frac{31}{2} \right\rceil = 16.$$

Branch a, e :

The branch forces edge $ae = 8$. Therefore, ae must be counted for city a and city e .

City	two edges used in bound	s_i
a	$ac = 1, ae = 8$	$1 + 8 = 9$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$ec = 2, ea = 8$	$2 + 8 = 10$

$$LB = \left\lceil \frac{9 + 9 + 3 + 7 + 10}{2} \right\rceil = \left\lceil \frac{38}{2} \right\rceil = 19.$$

Why the Branch a, c Is Not Used

The branch a, c is not invalid. It is skipped only to avoid duplicate tours.

In an undirected TSP, a tour and its reverse are the same cycle. For example,

$$a \rightarrow c \rightarrow e \rightarrow d \rightarrow b \rightarrow a$$

has the same edges as the reversed tour

$$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a.$$

So a tour that begins with $a \rightarrow c$ can often be represented in reverse as a tour that begins with $a \rightarrow b$. To avoid generating both versions, the search fixes a direction/order and skips a, c at the first level.

Key Point

a, c is valid, but unnecessary in this organized search.

8 Completing Candidate Tours

After expanding the promising branch a, b , the tour must still visit the remaining cities

$$\{c, d, e\}$$

and then return to the starting city a .

So every candidate in this part has the form

$$a \rightarrow b \rightarrow \text{remaining cities in some order} \rightarrow a.$$

For example,

$$a \rightarrow b \rightarrow c \rightarrow e \rightarrow d \rightarrow a$$

means the selected edges are

$$ab, \quad bc, \quad ce, \quad ed, \quad da.$$

The length is computed by adding the edge weights from the graph.

Step-by-Step Search Under a, b

After $a \rightarrow b$, the next city can be c , d , or e . The search compares these live branches by their lower bounds.

Partial path	Meaning	Bound/status
a, b, c	continue through city c	$LB = 16$
a, b, d	continue through city d	$LB = 16$
a, b, e	continue through city e	$LB = 19$

How These Bounds Are Calculated

For a partial path, all edges already in the path are forced. The lower-bound calculation must include those forced edges at their endpoint cities. Each city still needs two incident edges in the final tour.

Path a, b, c :

The forced edges are

$$ab = 3, \quad bc = 6.$$

City	two edges used in bound	s_i
a	$ab = 3, ac = 1$	$3 + 1 = 4$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$cb = 6, ca = 1$	$6 + 1 = 7$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

$$LB(a, b, c) = \left\lceil \frac{4 + 9 + 7 + 7 + 5}{2} \right\rceil = \left\lceil \frac{32}{2} \right\rceil = 16.$$

Path a, b, d :

The forced edges are

$$ab = 3, \quad bd = 7.$$

City	two edges used in bound	s_i
a	$ab = 3, ac = 1$	$3 + 1 = 4$
b	$ba = 3, bd = 7$	$3 + 7 = 10$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$db = 7, de = 3$	$7 + 3 = 10$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

$$LB(a, b, d) = \left\lceil \frac{4 + 10 + 3 + 10 + 5}{2} \right\rceil = \left\lceil \frac{32}{2} \right\rceil = 16.$$

Path a, b, e :

The forced edges are

$$ab = 3, \quad be = 9.$$

City	two edges used in bound	s_i
a	$ab = 3, ac = 1$	$3 + 1 = 4$
b	$ba = 3, be = 9$	$3 + 9 = 12$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$eb = 9, ec = 2$	$9 + 2 = 11$

$$LB(a, b, e) = \left\lceil \frac{4 + 12 + 3 + 7 + 11}{2} \right\rceil = \left\lceil \frac{37}{2} \right\rceil = 19.$$

The branches a, b, c and a, b, d are more promising than a, b, e , because

$$16 < 19.$$

If live branches have different lower bounds, choose the branch with the smallest lower bound. For example, if

$$LB(a, b, c) = 16, \quad LB(a, b, d) = 17, \quad LB(a, b, e) = 18,$$

then the next branch to expand would be

$$\boxed{a, b, c}$$

because

$$16 < 17 < 18.$$

If two branches have the same lower bound, either one may be expanded first. A common rule is to choose the branch that was generated first.

So in this example, a, b, c and a, b, d are tied:

$$LB(a, b, c) = LB(a, b, d) = 16.$$

The algorithm may expand either one first, but it should keep the rule consistent.

Later, when a complete tour of length 16 is found, the branch a, b, e is no longer worth expanding because

$$LB(a, b, e) = 19 \geq 16 = \text{bestTourLength}.$$

Why $a \rightarrow b \rightarrow e$ Is Not Continued

$a \rightarrow b \rightarrow e$ is valid, but its lower bound 19 cannot beat the best tour length 16.

Next Step: Expand a, b, c and a, b, d

After a, b, e is set aside, the useful live branches are

$$a, b, c \quad \text{and} \quad a, b, d.$$

When to Recalculate LB

If many cities are still unvisited, compute a new lower bound for the partial path and compare it with the current best tour length.

If only one or two cities are left, it is usually clearer to finish the possible tours directly and calculate their actual lengths instead of estimating another bound.

Here, after a, b, c or a, b, d , only two cities remain. Therefore, we complete the tours directly and compute their actual lengths.

Expand a, b, c :

The path already uses

$$a \rightarrow b \rightarrow c.$$

The remaining cities are

$$\{d, e\}.$$

There are only two possible completions:

Choice after a, b, c	Complete tour forced by the choice	Length
Go to d next	$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a$	$3 + 6 + 4 + 3 + 8 = 24$
Go to e next	$a \rightarrow b \rightarrow c \rightarrow e \rightarrow d \rightarrow a$	$3 + 6 + 2 + 3 + 5 = 19$

So under a, b, c , the better completion is

$$a \rightarrow b \rightarrow c \rightarrow e \rightarrow d \rightarrow a$$

with length

$$19.$$

Expand a, b, d :

The path already uses

$$a \rightarrow b \rightarrow d.$$

The remaining cities are

$$\{c, e\}.$$

Again, there are only two possible completions:

Choice after a, b, d	Complete tour forced by the choice	Length
Go to c next	$a \rightarrow b \rightarrow d \rightarrow c \rightarrow e \rightarrow a$	$3 + 7 + 4 + 2 + 8 = 24$
Go to e next	$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$	$3 + 7 + 3 + 2 + 1 = 16$

So under a, b, d , the better completion is

$$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$$

with length

$$16.$$

Next-Step Conclusion

After expanding a, b, c and a, b, d , the best tour found has length 16.

Tour	Length	Status
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a$	$3 + 6 + 4 + 3 + 8 = 24$	first complete tour
$a \rightarrow b \rightarrow c \rightarrow e \rightarrow d \rightarrow a$	$3 + 6 + 2 + 3 + 5 = 19$	better tour
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow e \rightarrow a$	$3 + 7 + 4 + 2 + 8 = 24$	inferior tour
$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$	$3 + 7 + 3 + 2 + 1 = 16$	best tour

Length Calculations

Tour	Edge-by-edge calculation	Length
a, b, c, d, e, a	$ab + bc + cd + de + ea = 3 + 6 + 4 + 3 + 8$	24
a, b, c, e, d, a	$ab + bc + ce + ed + da = 3 + 6 + 2 + 3 + 5$	19
a, b, d, c, e, a	$ab + bd + dc + ce + ea = 3 + 7 + 4 + 2 + 8$	24
a, b, d, e, c, a	$ab + bd + de + ec + ca = 3 + 7 + 3 + 2 + 1$	16

Updating the Best Tour

When the first complete tour is found,

$$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a,$$

its length is 24, so initially

$$\text{bestTourLength} = 24.$$

Then the next complete tour has length 19, so it improves the current best value:

$$19 < 24.$$

Finally,

$$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$$

has length 16, and

$$16 < 19.$$

Therefore, the current best tour is updated to length 16.

The best complete tour found is

$$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$$

with length

$$16.$$

9 Pruning

For minimization problems, the pruning rule is:

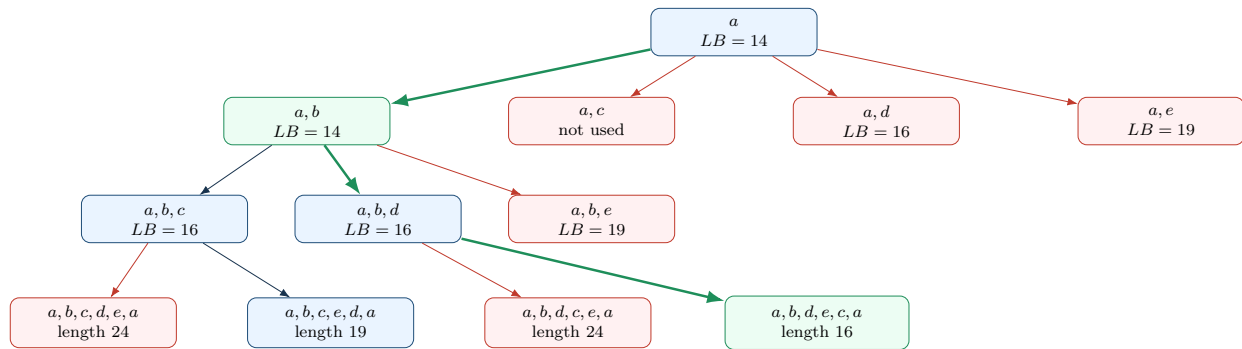
Pruning Rule

$$LB \geq \text{bestTourLength} \implies \text{prune the node.}$$

Once a tour of length 16 is found, every live branch with lower bound at least 16 cannot improve the answer. Such branches can be terminated.

Node	Bound or length	Decision
a, d	$LB = 16$	prune
a, e	$LB = 19$	prune
a, b, e	$LB = 19$	prune
a, b, d, e, c, a	length 16	incumbent optimal tour

10 State-Space Tree



11 Pseudocode

TSP Branch and Bound

```
TSPBranchAndBound(graph):
    fix the starting city as a
    bestTourLength = infinity
    bestTour = none

    create the root node with path [a]
    compute root.lowerBound
    liveNodes = {root}

    while liveNodes is not empty:
        node = remove the live node with smallest lowerBound

        if node.lowerBound >= bestTourLength:
            continue

        if node represents a complete tour:
            if node.length < bestTourLength:
                bestTourLength = node.length
                bestTour = node.tour
            continue

        create children by adding one allowed unvisited city

        for each child:
            compute child.lowerBound
            if child.lowerBound < bestTourLength:
                add child to liveNodes

    return bestTour and bestTourLength
```

12 Correctness Intuition

Branch and Bound does not ignore a branch randomly. A branch is pruned only when its lower bound proves that no completion of that partial tour can be better than the best complete tour

already found.

Therefore, if a branch could contain a better tour, its lower bound would be smaller than the current best tour length, and the algorithm would keep it live.

13 Complexity

Even with pruning, the worst case may still require checking many permutations of the cities. With one starting city fixed, the number of possible tours is proportional to

$$(n - 1)!.$$

For an undirected symmetric TSP, reverse tours are the same, so the number of distinct tours is

$$\frac{(n - 1)!}{2}.$$

Worst-case time is factorial.

Branch and Bound is useful because good bounds can greatly reduce the number of tours explored in practice.

14 Final Review

TSP Branch and Bound Essentials

TSP asks for a shortest Hamiltonian circuit.

$$LB = \left\lceil \frac{\sum s_i}{2} \right\rceil, \quad s_i = \text{sum of two smallest incident edges at city } i.$$

For minimization, prune when $LB \geq \text{bestTourLength}$.

Best tour here: $a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$.

Minimum tour length = 16.